



TOLNA VÁRMEGYEI SZC
APÁCZAI CSERE JÁNOS TECHNIKUM
ÉS KOLLÉGIUM

DOKUMENTÁCIÓ

Biró Szabolcs Benjámín

Halmai Bence Ádám

Magyar Barnabás

Informatika és távközlés

Szoftverfejlesztő és -tesztelő

5 0613 12 03

Dombóvár, 2026.04.30.

Tartalom:

1	Feladat rövid ismertetése	3
1.1	Bevezetés	3
1.2	Cél.....	3
1.3	Jövőbeli tervek	3
2	Használt technológiák.....	4
2.1	Programozási nyelvek, keretrendszerek	4
2.1.1	Frontend	4
2.1.2	Backend	4
2.2	Adatbázis.....	4
2.3	Egyéb technológiák	5
3	Adatbázis.....	6
4	Tesztelés	8
4.1	Tesztelési fajták	8
5	Program működésének bemutatása	8
6	Fejlesztői dokumentáció.....	11
6.1	API rétegeinek felépítése és szerepkörei	11
6.1.1	Config Package.....	11
6.1.2	Controller Package	12
6.1.3	DTO Package	12
6.1.4	Entity Package	12
6.1.5	Exception Package.....	13
6.1.6	Repository Package	13
6.1.7	Service Package	13
6.2	Frontend leírása	13
6.2.1	Könyvtárstruktúra	13
6.2.2	Component-ek	14
6.2.3	Service-ek.....	15
6.2.4	Egyéb.....	15
6.3	A projekt üzemeltetése	15
7	Csapatmunka.....	15

7.1	Csapat bemutatása.....	15
7.2	Szerepek.....	15

1 Feladat rövid ismertetése

1.1 Bevezetés

A projekt az elavultnak mondható jogosítványszerzési folyamatot kívánja megújítani és egyben modernizálni. A webes felületnek köszönhetően az autósiskolák, a diákok és az oktatók egy egyszerű és letisztult felületet kapnak. A regisztrációtól kezdve az oktatóhoz és az iskolához történő csatlakozáson át egészen vezetési óra foglalásig a felhasználóknak lehetőségük nyílik a folyamatok kezelésére.

1.2 Cél

A Félúton célja, hogy a nehezen visszakövethető és bonyolult jogosítványszerzési folyamatot leegyszerűsítse. Az oldal már a jogosítvány szerzés első pontjától - az autósiskolához való csatlakozástól - kezdve szándékozik könnyíteni a leendő diákok életét. Továbbá nemcsak a diákok számára nyújt segítséget, hanem az oktató munkáját is könnyíti. A diák számára az óra nyomon követését célozza meg könnyíteni, az oktatónak továbbá a diákjai számontartását is leegyszerűsíti. Az iskolák számára az egy helyen történő tag kezelést is biztosítja. Ezekkel a funkciókkal szeretné a webes felület elérni azt, hogy a jogosítványszerzés egy sokkal egyszerűbb és nyomon követhetőbb folyamat legyen.

1.3 Jövőbeli tervek

Az oldalra szándékunkban áll bevezetni a vizsgák kezelését. Maguk a forgalmi vizsgák az autósiskoláktól függetlenek, ezért azt egy harmadik féltől származó forrásból kellene kezelni. Jelenleg a felhasználók egymással csak e-mailen keresztül tudnak kommunikálni. Ezt úgy szeretnénk majd javítani, hogy a felhasználók a weblapon belül tudjanak majd kommunikálni e-mail használata nélkül. Jelenleg a diákok az óráikat csak személyesen az adott autósiskolában tudják kifizetni. Ebből kifolyólag szeretnénk majd bevezetni az online fizetési lehetőséget is. Végül össze szeretnénk hozni, hogy egy új felhasználó a Facebook- vagy Google-fiókjával tudjon regisztrálni az oldalra.

2 Használt technológiák

2.1 Programozási nyelvek, keretrendszerek

A projekt során a következő programozási nyelveket, illetve keretrendszereket használtuk:

2.1.1 Frontend

A projekt frontend része Angular keretrendszerre épül. Választásunk oka a TypeScript-támogatás, a komponensalapú felépítés és a keretrendszerben lévő beépített támogatások voltak. Továbbá Bootstrap keretrendszert használtunk a reszponzivitás elérés érdekében. A projekt frontend része a következő nyelveket használja:

- HTML5
- CSS3
- Typescript

2.1.2 Backend

A frontend által használt REST API (Representational State Transfer Application Programming Interface) Spring Boot keretrendszerrel, Java alapon lett fejlesztve. A backend projekt a következő főbb függőségeket (dependencyket) tartalmazza:

- Spring boot actuator
- Spring boot data jpa
- Spring boot web
- Spring boot devtools
- MYSQL driver
- Spring boot test
- Validation
- Project lombok
- Open API
- Spring boot security
- Bouncycastle
- Spring boot emailsender

2.2 Adatbázis

Adatbázisunk MySQL segítségével felépített, strukturált relációs adatbázis. Csak logikai törlést tartalmaz, továbbá a futásidő optimalizálása miatt, a projekt által használt fájloknak csak az elérési útvonalát tartalmazza.

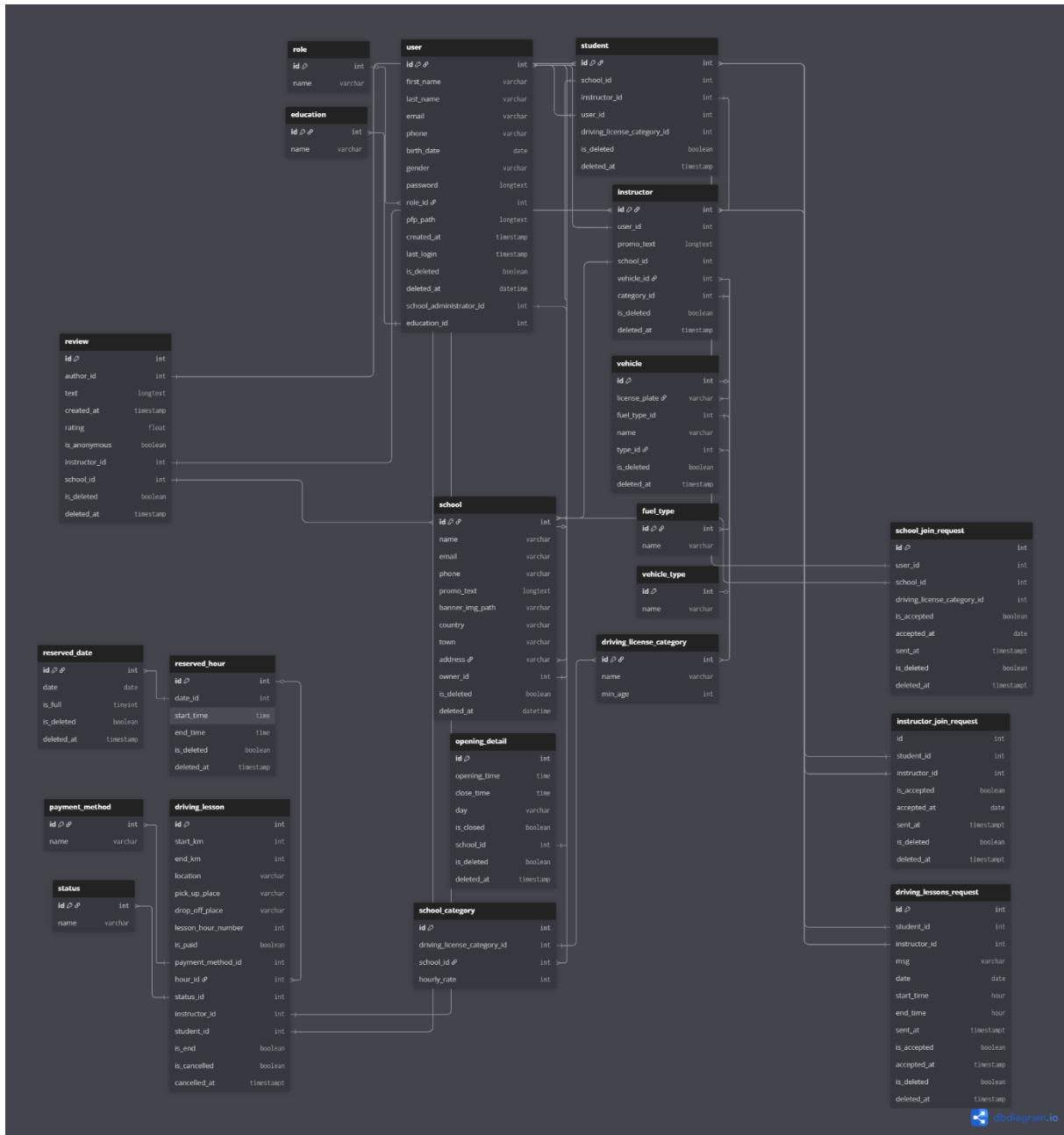
2.3 Egyéb technológiák

Az alábbi technológiákat, illetve szoftvereket főleg a csapatmunka lebonyolítására, illetve tervezésre használtuk.

- **Figma:**
 - A Figma egy online dizájnintervező szoftver és weboldal, amellyel egyszerre, egy időben több felhasználó tudja a dizájntervet szerkeszteni. Segítségével továbbá a reszponzív dizájnt is szerkeszteni lehet.
- **Git:**
 - A Git egy elosztott verziókezelő rendszer. Segítségével a fejlesztők nyomon követhetik a fájlok változásait, és könnyen együtt dolgozhatnak másokkal. A Git lehetővé teszi a különböző verziók közötti váltást, valamint az ágak (branchek) kezelését is. Gyors, hatékony és széles körben használják szoftverfejlesztési projektekből.
- **Jira:**
 - A Jira egy projektmenedzsment és hibakövető rendszer. Segítségével a csapatok nyomon követhetik a feladatokat, hibákat és fejlesztési folyamatokat. Különösen népszerű az agilis módszertanok – például a Scrum és a Kanban – támogatásában. Rugalmasan testreszabható, így különböző típusú projektekhez is jól alkalmazható.

3 Adatbázis

A projekt által használt adatbázis az OOP (Object Oriented Programing) elvét követi, így még jobban szinkronban van az adatbázis és az API-nak alapot szolgáló backend.



1. ábra, Az adatbázis terv

Az alábbi kép ábrázolja az általunk használt adatbázis ER (Entity-Relationship) diagramját. Az alábbi diagramon látható az adatbázisban tárolt táblák (entitások), attribútumok és kapcsolatok (relációk). Az összes tábla rendelkezik elsődleges kulccsal (Primary Key), és az egymásra való hivatkozás idegen kulccsal (Foreign key) valósul meg.

A diagram a következő részekre bontható fel:

- **Felhasználó adatok:** A felhasználó főbb adatait a user tábla tartalmazza. A felhasználónak többek között a nevét, jelszavát, telefonszámát, e-mail-címét és iskolai végzettségét tartalmazza. Továbbá a felhasználó rangját a role tábla tartalmazza. Ha a felhasználó diák vagy oktató akkor a redundancia elkerülése érdekében külön táblában a student/instructor táblában tároljuk a további adatokat.
- **Iskola adatai:** Az iskola adatait, mint például az iskola nevét, az iskola címét és az iskola telefonszámát tartalmazza. Az iskolának továbbá a tulaját is tároljuk hivatkozással.
- **Oktató:** Az oktató alapadatait az instructor tábla tartalmazza. Az oktatónak továbbá egy járműve is van. A jármű adatait többek között a vehicle, a vehicle_type és a fuel_type tábla tartalmazza.
- **Vezetési óra:** A vezetési óra adatait a driving_lesson tábla tartalmazza. Egy óráról a km-óra állását kezdéskor/befejezéskor, a felvevés és letétel helyét, az időpontot és az óraszámot tároljuk. Az időpont tárolása a reserved_hour és reserved_date tábla biztosítja.
- **Kérelmek:** A kérelmekből 3 különböző fajta van jelen. Ezek a school_join_request, driving_lesson_request és az instructor_join_request táblák. A táblák a feladó és a címzett ID-ját tartalmazzák, továbbá egy status adatot, amely az elfogadást jeleníti meg. A driving_lesson_request továbbá a vezetési óra időpontját is tartalmazza.

4 Tesztelés

A tesztelés során három különböző fajta tesztelés ment végbe.

4.1 Tesztelési fajták

- **Manuális tesztelés:**
 - A manuális tesztelés backenden és frontenden is történt. Frontenden Google Chrome segítségével történt. Backenden a manuális teszt Postman segítségével történt.
- **Unit tesztelés:**
 - Unit tesztelés backenden történt a Java alapú JUnit segítségével. Az API validációs része volt ezzel a módszerrel tesztelve.
- **Integrációs tesztelés:**
 - A backend endpointjainak a tesztelésére integrációs tesztek voltak használva. Az integrációs tesztet Java alapon mockolással, Mockitoval értük el.

5 Program működésének bemutatása

A weblap hat különböző felhasználói profilra épül:

- Felhasználó
- Diák
- Oktató
- Iskola adminisztrátor/Iskola tulajjal
- Oldal adminisztrátor

Az oldalon megjelenő felületek mind adott ranghoz, jogosultsághoz kötöttek, és ez alapján fog megjelenni mindig a tartalom. Az oldal funkciói felhasználói profilhoz kötöttek. Regisztrált és bejelentkezett profil nélkül a felhasználó semmit sem tud elérni az oldalon. Továbbá jogosultságtól függetlenül a felhasználó a saját profilját szerkeszteni és törölni viszont tudja.

Felhasználói jogosultság:

A bejelentkezés után ezt a jogosultságot kapja meg a felhasználó. Ezzel a jogosultsággal a felhasználó csak iskolát tud keresni, és ahhoz csatlakozási kérelmet tud csak küldeni.

Diák jogosultság:

A diák jogosultságot a felhasználó csak egy elfogadott iskola csatlakozási kérelem után kaphatja meg. Ezzel a jogosultsággal a következő funkciót nyílnak meg a felhasználó számára:

- Oktatóhoz való csatlakozási kérelem küldése.
- Oktatóról/Iskoláról való véleménynyilvánítás
- Vezetésióra-kérelem küldése. Ezt csak akkor tudja megtenni a felhasználó, ha már egy oktató diákja.

Oktató jogosultság

Az oktatói rangot csak akkor kapja meg a felhasználó, ha a regisztráció során a „Regisztrálás mint” résznél az oktatót választja ki. Ezzel a jogosultsággal a következő funkciók nyílnak meg a felhasználó számára:

- Iskolához való csatlakozási kérelem küldése.
- Diákcsatlakozási kérelem elbírálása.
- Vezetésióra-kérelem elbírálása.
- Hozzá tartozó jármű adatainak szerkesztése.
- Diákjai kezelése (diák eltávolítása).
- Vezetési óra lemondása.
- Vezetési óra adatainak szerkesztése.

Iskola adminisztrátor/Iskola tulaj jogosultság

Az iskola adminisztrátora és az iskola tulajdonosa funkciók szerint nagyon hasonlóak, egyedül az iskola tulajdonosának kettővel több funkciója van. Az iskolatulajdonos jogosultságot akkor kaphatja meg a felhasználó, ha az oldal adminisztrátora az ő profilját jelöli ki az iskola létrehozásakor.

Az iskola tulajának a következő funkciói vannak:

- Iskola törlése
- Iskolaadminisztrátorok beállítása

Az iskola adminisztrátorának és az iskola tulajának a következő funkciók érhetőek el:

- Iskola tagjainak kezelése
- Iskola adatainak szerkesztése (ebben benne van a nyitvatartás is)
- Csatlakozási kérelmek elbírálása

Az oldal adminisztrátorai a következő funkciókkal rendelkeznek:

- Felhasználók kezelése
- Iskolák kezelése
- Iskolák létrehozása

6 Fejlesztői dokumentáció

6.1 API rétegeinek felépítése és szerepkörei

Az API rétegezettsége törekszik arra, hogy a lehető legletisztultabb és legegyszerűbb legyen.

A következő rétegek vannak jelen az API-ban:

6.1.1 Config Package

Szerep: A config package-ben lévő osztályok felelősek a projekt konfigurációs beállításaiért

Funkciók:

- **Biztonság beállítása:** A **security** package tartalmazza az API védelméért felelős osztályokat. Ezek az osztály állítják be a végpontok védelmek illetve a JWT alapú autentikációt/autorizációt. Az utóbbit pontosabban a JWT package-ben lévő osztályok kezelik le.
- **Email küldés beállítása:** Az **email** package tartalmazza az email küldéséhez szükséges konfigurációs classokat. Ezek a classok felelősek az email küldéséért, illetve itt jön létre az ehhez szükséges HTML template-eket.

Felépítése:

- email
 - EmailConfiguration
 - EmailSender
 - EmailTemplateConfiguration
- security
 - JWT:
 - RefreshToken
 - RefreshToken
 - JWTGeneratorFilter
 - JWTPropertyConfiguration
 - JWTUtilsService
 - JWTValidatorFilter
 - SecurityConfig
 - UserSetter

6.1.2 Controller Package

Szerep: A http kérések fogadása és azok továbbítása a service réteg felé. Illetve a Swagger dokumentáció annotációi itt vannak jelen.

Funkciók:

- **HTTP kérések fogadása:** A bejövő HTTP kéréseket itt fogadja az API.
- **Kérések feldolgozása:** A Controller réteg felelős a kapott adatok kinyeréséért, illetve azok továbbításáért a service réteg felé. Itt történik a requestBody, a requestParamok és pathVariable-k kinyerése.
- **Válaszok elküldése:** A továbbítás után ez a réteg felelős a kapott kérésre adott válasz küldéséért.

RestController osztályok és felelőségük:

- DrivingLessonController: A vezetési órákhoz kapcsolódó http kéréseket kezeli.
- InstructorController: Az oktatóhoz kapcsolódó http kéréseket kezeli.
- OtherStuffController: Az egyéb dolgokhoz kapcsolódó http kérések kezelése, mint például a tankolási típusok, városok, fizetési módszerek.
- RequestController: A kérelmek küldéséhez kapcsolódó http kérések kezelése
- ReviewController: Az értékelésekhez kapcsolódó http kérések kezelése
- SchoolController: Az iskolákhoz kapcsolódó http kérések kezelése
- StudentController: A diákokhoz kapcsolódó http kérések kezelése
- UserController: A felhasználókhöz kapcsolódó http kérések kezelése

6.1.3 DTO Package

Szerepe: Tartalmazza az API által használt összes DTO (Data Transfer Object) record típusú osztályt.

Funkciója: A rétegek között az adatokat továbbítja, ezzel növelve az adatbiztonságot. Továbbá a testreszabhatósága komoly segítséget nyújt.

6.1.4 Entity Package

Szerepe: Az API által használt adatbázis tábláit megjelenítő entitás osztályok. Felépítésük megegyezik az adatbázisban lévő táblákhoz.

6.1.5 Exception Package

Szerepe: Egyedi exception-ok létrehozásával az üzleti logika során létrejövő hibák jelzése és lekezelése. Egy globális exception handler segítségével a futás idő alatt létrejövő exceptionokat kezeli le és ad vissza az adott exception-re beállított választ.

Egyedi exception-ok:

- `InvalidDataException`: Egy bizonyos adat nem felel meg bizonyos validációs szabályoknak. 415-ös hibakódot küld vissza a felhasználónak.
- `NotFoundException`: Olyan rekord lekérésekor merül fel ez a hiba, amikor nem létező adatot szeretne a felhasználó lekérni az adatbázisból. 404-es hibakódot küld vissza a felhasználónak.
- `UniqueErrorException`: Olyan adat felvitelekor történik meg, amely már regisztrált adat az adatbázisban. 409-es hibakódot küld vissza a felhasználónak.

6.1.6 Repository Package

6.1.7 Service Package

6.2 Frontend leírása

6.2.1 Könyvtárstruktúra

A projektünk frontend részénél törekedtünk egy letisztult és átlátható könyvtárstruktúrát létrehozni. A következő könyvtárstruktúrát használjuk a frontend résznél:

- `src`:
 - `app`
 - `components`
 - `interceptor`
 - `models`
 - `pipe`
 - `routeGuards`
 - `services`
 - `assets`: A weblap által használt statikus fájlokat tartalmazó mappa

6.2.2 Component-ek

Az alkalmazást funkciók, nézet és jogosultság szerint componentekre osztjuk. Az alkalmazást a következő componentekre osztottuk:

- **about:** A rólunk részleget megjelenítő component
- **calendar:** Az oktató és diák számára megjelenő naptár részleg. Ezen a componenten kezelheti a felhasználó az óráit.
- **driving-lesson-editor:** Az oktató ezen a componenten tudja az óra adatait szerkeszteni.
- **request-container:** A diák ezen a componenten tudja elküldeni a vezetési órához kapcsolódó kérelmet.
- **faq:** A gyakori kérdések felületének componense
- **footer:** A weboldal footer-jének a componense. További linkeket tartalmaz az oldalon belülrre, vagy kívülrre.
- **homepage:** A főoldal, a főbb tartalmakhoz előbb bejelentkezés szükséges, és csak azután jelenik meg a fő tartalom.
- **login-page:** Bejelentkező oldal.
- **navbar:** Az oldal navigációs sávja, rendes képernyős nézetben,
- **navbar-phone:** Az oldal navigációs sávja, tablet vagy telefon nézetben.
- **profil-card:** Egy adott profilhoz kapcsolódó kártya.
- **profil-page:** A felhasználói oldal, ezen érhető el az adott felhasználó vagy iskola oldala. Az adott profillal kapcsolatos műveleteket innen lehet elindítani.
- **admin-setter:** Az iskolának az adminisztrátorai itt állíthatóak be.
- **price-list:** Az adott iskola árlistáját megjelenítő componens.
- **profil-editor:** Egy adott profil szerkesztési felülete. Tartalma a profil típusa, illetve jogosultsága szerint változik
- **review-card:** Egy adott véleményt tartalmazó kártya
- **review-list:** Egy adott oktatóról/iskoláról szóló véleményeket kilistázó component.
- **review-writer:** Vélemény író felületet biztosító component.
- **registration-page:** Regisztráció felületet tartalmazó component.
- **request-list:** Az adott iskolához/oktatóhoz küldött kérelmeket kilistázó component
- **request-card:** Egy adott kérelem adatait tartalmazó component.
- **school-registration:** Iskola regisztrációs felületét tartalmazó component.
- **search-page:** Az oldal keresőfelülete, a felhasználó oktatót vagy iskolát tud keresni rajta.
- **unauthorized:** Az oldal azon része, amelyik akkor jelenik meg, ha egy felhasználó olyan részlegre megy az oldalnak, amihez nincs jogosultsága
- **user-list**

6.2.3 Service-ek

A projekt Frontend részén lévő service-k a backenden lévő service-kre alapszik. Az eltérés köztük, hogy egyser service frontenden tartalmaz több component által használt változókat is. Továbbá az **alert-service** felelős az adott események utáni értesítő ablak működéséért.

6.2.4 Egyéb

A projekt frontend részre továbbá a következő harmadik féltől származó package-eket tartalmazza:

- angular material
- ngx cookie-service

6.3 A projekt üzemeltetése

7 Csapatmunka

7.1 Csapat bemutatása

A csapat még 2025 tavaszán alakult meg. A csapat összetétele azért alakult így, mert 3an tökéletesen kiegészítjük egymás gyengeségét. A csapatot Biró Szabolcs Benjámin, Halmai Bence Ádám és Magyar Barnabás alkotja.

7.2 Szerepek

A csapattagoknak a következő szerepei vannak:

- **Biró Szabolcs Benjámin:** Szabolcs a projektben a frontend fejlesztő, illetve a designer volt. Továbbá a projekthez kapcsolódó prezentációkat és dokumentumokat is ő formázta, dizájnolta meg.
- **Halmai Bence Ádám:** Bence a projektben a backend fejlesztő és a project manager. Továbbá közreműködött a design terv készítésében és a projekthez kapcsolódó dokumentumokat és prezentációkat ő állította össze.
- **Magyar Barnabás:** Barnabás a projektben az adatbázis kezelő volt. Az adatbázis kezelés mellett, az ő felelősége volt a további ügyletek elvégzése, mint például a projektmunka alatt használt kommunikációs tér létrehozása, valamint a projekt által használt e-mail cím létrehozása. Továbbá a mobil nézet dizájntervét ő készítette el.